Lab 02: Advanced Dax

Lab 02: Advanced DAX

Lab summary

These labs build a trusted DAX measure layer over the Lab 01 semantic model.

Azure Government readiness

The required labs are **Gov-ready** because they use Power BI Desktop and core DAX. DAX Studio and other external-tool diagnostics are **Verify for Gov**.

Prerequisites

- Completed Lab 01 semantic model or equivalent model.
- Tables named similarly to `FactSales`, `FactTargets`, `DimOrderDate`, `DimCustomer`, `DimProduct`, `DimTerritory`, `DimSegment`, and `BridgeCustomerSegment`.

Novice-friendly how-to guide

Create a measure

1. In Power BI Desktop, open **Report view** or **Model view**.
2. Select the table where the measure should live.
3. Select **Modeling > New measure**.
4. Type the measure name, equals sign, and DAX expression, such as `Sales Amount = SUM(FactSales[SalesAmount])`.
5. Press Enter.
6. Use the **Measure tools** ribbon to set the format.
7. Test the measure in a card, table, or matrix before using it in another measure.

Test DAX in a visual

1. Add a table or matrix visual.
2. Add one dimension field such as `DimCustomer[CustomerName]`.
3. Add the measure being tested.
4. Add slicers for date, territory, or product category.
5. Change slicer selections and confirm the measure responds as expected.

Create a calculation group

1. Confirm base measures such as `[Sales Amount]` and `[Gross Margin]` work before adding advanced logic.
2. In Power BI Desktop, open **Model view**.
3. On the ribbon, select **Calculation group**.
4. If prompted, enable **Discourage implicit measures**. Calculation groups require explicit measures.
5. Rename the calculation group table to `Time Intelligence`.
6. Rename the calculation group column to `Time Calculation`.
7. Rename the first calculation item to `Current` and set its expression to `SELECTEDMEASURE()`.
8. Add more calculation items from **Model Explorer**.
9. Test the calculation group in Power BI Desktop with `DimOrderDate` and a base measure.

Exercise 1: Row context vs. filter context

Objective: Diagnose why measures evaluate differently in different visuals.

Tasks

1. Create base sales, quantity, and gross margin measures.
2. Add a table visual by customer and product category.
3. Add slicers for territory and date.
4. Observe how slicers and visual rows change filter context.
5. Create a calculated column example only to demonstrate row context, then discuss why measures are preferred for aggregations.

Expected result

Learners can explain why the same measure returns different values across visual cells, slicers, and totals.

Exercise 2: Context transition and `CALCULATE`

Objective: Use `CALCULATE` to intentionally modify filter context.

Tasks

1. Create `[Sales Amount]`.
2. Create a measure that removes product filters.
3. Create product sales share using the unfiltered product total.
4. Create a measure with `KEEPFILTERS` for a selected customer type.
5. Compare results in visuals with slicers applied.

Expected result

Learners can explain filter replacement, filter removal, and filter preservation.

Concept note: choosing filter modifiers

Use `REMOVEFILTERS` when a measure should ignore one table or column, `ALL` when you also need the returned table behavior for iteration or ranking, `ALLEXCEPT` when a calculation should clear most filters but preserve a named grain, and `TREATAS` when a disconnected selection must be applied to a model column. Test each pattern in a simple visual before using it in a production measure.

Exercise 3: Advanced time intelligence

Objective: Create reusable time-intelligence measures.

Tasks

1. Validate the order date table.
2. Create Sales YTD.
3. Create Sales Prior Year.
4. Create Sales YoY and Sales YoY %.
5. Create a rolling 90-day sales measure.
6. Validate results by month.

Expected result

Learners can branch time-intelligence measures from a trusted base measure.

Exercise 4: Semi-additive measures

Objective: Understand measures that should not be summed across time.

Tasks

1. Review the difference between transaction facts and snapshot facts.
2. Discuss examples such as inventory, backlog, headcount, or account balance.
3. Create or review an ending-balance-style pattern using the DAX pattern reference.
4. Explain why summing snapshots across dates produces incorrect answers.

Expected result

Learners can identify semi-additive scenarios and choose an appropriate last-value or last-nonblank pattern.

Concept note: measure branching and naming

Keep base measures short, formatted, and reusable. Build derived measures such as variance, share, time intelligence, and rankings from those base measures instead of repeating `SUM` expressions. Consistent names and simple test visuals make it easier to spot context mistakes before a measure is reused across pages.

Exercise 5: Dynamic Top N and ranking

Objective: Rank customers and filter visuals to a dynamic Top N view.

Tasks

1. Create a customer sales rank measure.
2. Create an `Is Top 5 Customer` flag measure.
3. Add a customer bar chart.
4. Filter the visual to Top 5 customers.
5. Compare `ALL`, `ALLSELECTED`, and visual filters.

Expected result

Learners can build rankings that respect the intended report context.

Exercise 6: Dynamic titles and measure switching

Objective: Make report text and metrics respond to user selections.

Tasks

1. Create a dynamic title measure using selected territory or region.
2. Create a disconnected `MetricSelector` table.
3. Create a selected metric measure using `SWITCH`.
4. Add a slicer for metric selection.
5. Validate formatting and fallback behavior.

Expected result

Learners can create guided flexibility without duplicating pages or visuals.

Exercise 7: Calculation groups for reusable time intelligence

> **Azure Government note:** This lab is marked **Verify for Gov**. Confirm the target Power BI Desktop version exposes native calculation group authoring. TMDL View, Git/CI-CD workflows, XMLA, external tools such as Tabular Editor, capacity behavior, and customer workstation policy also require validation before making those paths required.

Objective: Reduce repeated time-intelligence measures using a calculation group.

Tasks

1. Confirm base measures for sales and gross margin work.
2. Use Power BI Desktop Model view to add a `Time Intelligence` calculation group.
3. Add calculation items for Current, Prior Year, Year-over-Year Change, and Year-over-Year Change %.
4. Optional: review the equivalent TMDL View pattern.
5. Optional: use Tabular Editor only when external tooling is validated.
6. Test calculation items against multiple base measures.

Option 1: Create the calculation group in Power BI Desktop

1. Open the semantic model in **Model view**.
2. Select **Calculation group** from the ribbon.
3. If Power BI prompts you to enable **Discourage implicit measures**, accept the prompt. Calculation groups work with explicit measures, not implicit drag-and-drop aggregations.
4. Rename the calculation group table to `Time Intelligence`.
5. Rename the calculation group column to `Time Calculation`.
6. Rename the first calculation item to `Current`.
7. Set the `Current` expression to `SELECTEDMEASURE()`.
8. In **Model Explorer**, add calculation items for `Prior Year`, `YoY Change`, and `YoY Change %`.
9. Use the expressions in the table below.
10. Add `Time Intelligence[Time Calculation]` to a slicer, matrix column, or other visual well.
11. Test the calculation group against `[Sales Amount]` and `[Gross Margin]`.

 Annotated calculation group in Power BI Desktop Model view

Callout	What it shows	How it maps to the steps
1	**Model** tab in the Data pane	Confirms you are working in the semantic model metadata, not only report fields.
2	**Calculation groups** node	Shows the calculation group container created by the ribbon command.
3	`Name` property	Rename the calculation group table to `Time Intelligence`.
4	**Calculation group column**	Rename this column to `Time Calculation`; this is the field users place in slicers or visuals.
5	**Calculation items** folder	Use this area in Model Explorer to add more calculation items.
6	First calculation item	Rename the initial item to `Current` and set its expression to `SELECTEDMEASURE()`.

Option 2: Create or review the calculation group in TMDL View

Use this option when learners are already working with PBIP, Git integration, Fabric, or CI/CD workflows and the environment supports TMDL editing.

```
createOrReplace

table 'Time Intelligence'

    calculationGroup
        precedence: 20

        calculationItem 'Current' =
            SELECTEDMEASURE()

        calculationItem 'YTD' =
            CALCULATE(
                SELECTEDMEASURE(),
                DATESYTD(DimOrderDate[Date])
            )
```

After applying TMDL changes, return to Power BI Desktop and validate the generated calculation group in a visual.

Option 3: Use Tabular Editor when approved

1. Open **External tools > Tabular Editor**.
2. Create a calculation group named `Time Intelligence`.
3. Rename the column to `Time Calculation`.
4. Add the calculation items from the table below.
5. Save the model and test in Power BI Desktop.

Calculation group answer key

Use these formulas for the `Time Calculation` calculation items. Each item uses `SELECTEDMEASURE()` so the same time-intelligence logic can be applied to `[Sales Amount]`, `[Gross Margin]`, `[Quantity]`, or another explicit base measure.

Current

```
SELECTEDMEASURE()
```

MTD

```
CALCULATE (
    SELECTEDMEASURE(),
    DATESMTD ( DimOrderDate[Date] )
)
```

QTD

```
CALCULATE (
    SELECTEDMEASURE(),
    DATESQTD ( DimOrderDate[Date] )
)
```

YTD

```
CALCULATE (
    SELECTEDMEASURE(),
    DATESYTD ( DimOrderDate[Date] )
)
```

Fiscal YTD

```
CALCULATE (
    SELECTEDMEASURE(),
    DATESYTD ( DimOrderDate[Date], "6/30" )
)
```

Prior Year

```
CALCULATE (
    SELECTEDMEASURE(),
    SAMEPERIODLASTYEAR ( DimOrderDate[Date] )
)
```

`YoY Change`

```
VAR PriorYearValue =
    CALCULATE (
        SELECTEDMEASURE(),
        SAMEPERIODLASTYEAR ( DimOrderDate[Date] )
    )
RETURN
    SELECTEDMEASURE() - PriorYearValue
```

`YoY Change %`

```
VAR PriorYearValue =
    CALCULATE (
        SELECTEDMEASURE(),
        SAMEPERIODLASTYEAR ( DimOrderDate[Date] )
    )
RETURN
    DIVIDE ( SELECTEDMEASURE() - PriorYearValue, PriorYearValue )
```

`Rolling 90 Days`

```
VAR LastVisibleDate =
    MAX ( DimOrderDate[Date] )
RETURN
    CALCULATE (
        SELECTEDMEASURE(),
        DATESINPERIOD ( DimOrderDate[Date], LastVisibleDate, -90, DAY )
    )
```

Conceptual alternate path

If Tabular Editor or XMLA workflows are blocked, compare the calculation group design to creating separate DAX measures such as `[Sales Prior Year]`, `[Gross Margin Prior Year]`, `[Sales YoY Change]`, and `[Gross Margin YoY Change]`.

Expected result

Learners understand when calculation groups improve DAX maintainability and why they require tenant/tool validation.

Exercise 8: DAX optimization

Objective: Improve readability and reduce repeated calculations.

Tasks

1. Identify a measure with repeated logic.
2. Rewrite it using variables.
3. Branch from base measures instead of duplicating aggregation logic.
4. Test the optimized measure in a simple visual.
5. Optional: use DAX Studio or Performance Analyzer to compare behavior.

> **Azure Government note:** DAX Studio is marked **Verify for Gov**. Validate customer workstation policy, external tool usage, Service connectivity, and XMLA settings before making this a required lab step.

Expected result

Learners can improve measure maintainability and know when external diagnostics require validation.

Validation checklist

- [ ] Base measures exist and are formatted.
- [ ] Time-intelligence measures return expected values by month.
- [ ] Semi-additive pattern is explained or implemented.
- [ ] Ranking works at the intended filter scope.
- [ ] Dynamic title has a fallback value.
- [ ] Measure switch handles missing or multi-select cases.
- [ ] Calculation groups are implemented or reviewed conceptually as **Verify for Gov**.
- [ ] DAX optimization uses variables and measure branching.
- [ ] DAX Studio and external tooling are labeled **Verify for Gov**.